mimic.

PRODUCT REQUIREMENTS DOCUMENT

Record once. Replay forever.

A Chrome extension turns a single pass through a web task into a reusable automation — a fill-in-the-blanks form, a REST endpoint, and a marketplace product.

Document Version	`1.0.0`
Target Audience	`Indie automators, non-technical operators, automation resellers`
Repository	`github.com/adittosarkerr/mimic`
Primary Agent Context Files	`CLAUDE.md (repo root), this document`

1. Executive Summary & Business Case

1.1 Problem Statement

Anyone who repeats a task on a website — searching flights for a recurring trip, checking a hotel for a client, pulling structured listings from a marketplace, sending the same shaped email — either re-does the manual clicking every time, or has to write and maintain a custom scraper/bot per site. Custom scrapers break the moment a site changes its DOM, need constant credential handling, and are inaccessible to anyone who cannot write Playwright/Selenium code themselves.

1.2 Value Proposition

Mimic removes the coding step entirely. A Chrome extension records one real pass through a task — every click, keystroke, and navigation — with a resilient multi-signal selector fingerprint (id, CSS path, XPath, ARIA label, label text, data-testid, name, placeholder, frame path). The backend turns that single recording into an **AutomationSchema**: a clean, fill-in-the-blanks form synthesized by an LLM (with a deterministic heuristic fallback so the product never depends on one vendor being reachable). From there the same automation can be run three ways: a hosted server run (“go”), a run inside the user's own logged-in browser tab (“run in my browser”), or a REST API call authenticated by a per-account API key — so it is equally usable by a non-technical parent and by a script.

1.3 Business Value Metrics (KPIs)

Metric	Definition	Target
Recording-to-automation conversion	% of recordings that produce a usable AutomationSchema on first analyze	> 90%
Deterministic run rate	% of runs served by a direct site adapter (Booking/GoZayaan) vs. generic replay	Track per-site; grow via new adapters
Run success rate	% of “go” + “run in my browser” runs returning status: success	> 85% on adapter-covered sites
API adoption	Active API keys with ≥ 1 call in the trailing 30 days	Track post-launch
Marketplace liquidity	Listings with ≥ 1 sale	Track post-launch

2. Target User Personas

Persona A: “The Indie Automator” (Primary)

Profile: Runs the same travel/price search every week for personal or client use. Comfortable with a web form; not a developer.

Needs: Change the destination/dates and get a clean, scannable result list back — not a raw copy of the source page. Wants it to work the same way every single time.

Constraints: Zero tolerance for “works sometimes” automations; a flaky run is worse than no automation at all.

Persona B: “The Marketplace Seller”

Profile: Builds a well-tuned automation (e.g. a specific flight-route search) and wants to sell access to it to other users.

Needs: A listing flow, a fair, size-scaled platform fee, and a payout record they can trust.

Constraints: Needs a plan tier that unlocks selling; needs the underlying automation to be robust enough that buyers don't immediately refund.

Persona C: “The Developer Integrator”

Profile: Wants to call a Mimic automation from their own script or cron job — e.g. a nightly price check — without touching a browser at all.

Needs: A stable REST endpoint per automation, an API key they generate themselves, and a JSON response shape they can parse without guessing.

Constraints: Will not accept an API that silently returns HTML on error instead of JSON.

3. Product System Architecture & Data Schema

To keep an autonomous coding agent from re-deciding the architecture on every session, the system boundary is fixed and explicit:

```
[Chrome Extension - MV3, Vite]
   | records clicks/inputs (multi-signal selector fingerprint)
   | can ALSO replay in the user's own logged-in tab
   v
[RecordingSession JSON] --POST--> [Backend: Node + Express + Playwright]
                                    +- Inference      (LLM + heuristic fallback)
                                    +- Replay Engine  (headless -> stealth -> assisted)
                                    |    +- Deterministic Site Adapters
                                    |    +- Generic Event Replay + Extraction
                                    +- SaaS           (auth, plans, marketplace)
                                    +- REST API       (per-user keys, /api/v1/...)
                                             |
                                             v
                              [Postgres, any provider - or local JSON in dev]
   ^
   |
[Frontend - React + Vite]  dashboard . run . account . marketplace
```

3.1 Core Business Objects

The agent must treat these as the fixed vocabulary of the system — new features attach to these objects; they do not get redefined per feature.

Object	Purpose	Key Fields
RecordedEvent	One captured user action	type, selector fingerprint, value, url, timestamp
RecordingSession	The raw capture of one task	id, events[], segments[] (per-domain)
VariableField	One changeable input on the synthesized form	name, label, type, kind (input/choice/guests), autocomplete, sampleValue
AutomationSchema	The reusable, runnable product of a recording	automationId, title, variables[], startUrl, login, email
RunResult	One execution's outcome	runId, status, output.results[], stepsExecuted/Total, assisted, botWall
User / ApiKey	Account + REST credential	email, passwordHash (scrypt), plan, apiKeys[]
Listing / Transaction	Marketplace product + ledger entry	priceModel, priceBdt, platformFeeBdt, sellerNetBdt

3.2 Deterministic Adapter Contract

A site adapter is the highest-trust execution path: it builds the target site's own results URL directly from the form values (no click replay at all) and scrapes with fixed selectors, so identical inputs always produce identical navigation. An adapter must return `null` from `build()` whenever it cannot confidently resolve an input (e.g. an unmappable city) — the caller then falls back to generic replay. An adapter must never partially apply and silently produce a wrong result.

4. Comprehensive Feature Specifications (MoSCoW Matrix)

This is the section an autonomous agent parses to scope work. **Must** items are the pass/fail gate for “is this done.” **Won't** items are hard, negative instructions — they exist specifically to stop an agent from over-engineering, reverse-engineering a private API, or silently taking a shortcut that looks like progress but breaks trust (data loss, insecure storage, a crashed service).

4.1 MUST HAVE (Minimum Viable Product)

M.1 **Browser Recording Engine**

User Experience: The extension records a task exactly once, across any number of sites in one session, with no setup.

Functional Mechanics

- Every click/input/change/navigation is captured with a multi-signal selector fingerprint (id, css, xpath, ariaLabel, labelText, dataTestid, name, placeholder, framePath) so replay survives minor DOM changes.
- Recorded text uses layout-aware `innerText`, not raw `textContent`, to avoid concatenated-word corruption (e.g. “KualaLumpurMalaysia”).

M.2 Automated Form Synthesis

User Experience: After recording, the user presses “build automation” and gets a clean form — not a raw replay of clicks.

Functional Mechanics

- An LLM pass (DeepSeek) synthesizes the canonical title, description, and variable list from the full recording; a deterministic heuristic pass is the fallback so this step can never hard-fail.
- Shared-event fields (e.g. adults/children/rooms sharing one occupancy widget click; a synthesized check-out date sharing the only recorded calendar click) are modeled explicitly, not guessed at replay time.
- Live introspection reads real dropdown options and field bounds from the source site where reachable, and degrades to the recorded sample value where it is not (e.g. behind a bot wall).

M.3 Dual Replay Path

User Experience: The same automation runs identically whether the user presses “go” or “run in my browser.”

Functional Mechanics

- “go” executes server-side via an escalation ladder: headless (fast) → stealth-headed, off-screen, fresh throwaway profile (beats bot walls) → assisted, a visible window, only when a human must solve a CAPTCHA or log in.
- “run in my browser” replays the identical collapsed step list inside the user's own logged-in tab via the extension — required for sites that need an existing session (Gmail, etc).
- Both paths share one collapse/override engine so behavior never diverges between them.

M.4 Deterministic Site Adapters

User Experience: For known services, results are the same every run — never “works sometimes.”

Functional Mechanics

- Booking.com: hotels (city/dates/guests/star+breakfast+cancellation filters/currency), flights (via booking.kayak.com, IATA-resolved), attractions (city+country results page).
- GoZayaan: flights (IATA-resolved origin/destination, one-way and round-trip).
- Every adapter falls back to generic recorded-step replay if it cannot resolve an input — it must never guess and return a silently wrong destination or date.

Explicitly out of scope for direct-URL adapters: any service whose search requires an internal place-ID resolved only through that site's own autocomplete (Booking cars/attractions-without-city/taxis, GoZayaan hotels/tours/visa). These are served by hardened autocomplete replay instead — see W.2.

M.5 REST API with Per-User Keys

User Experience: A developer can call any automation from their own code with no browser involved.

Functional Mechanics

- POST /api/v1/automations/:id/run, authenticated with Authorization: Bearer mk_live_....
- Keys are generated/revoked from Account → API access; the full secret is shown exactly once at creation and only a masked form (mk_live_1a2b…f9c0) is ever stored or returned again.
- Every route — success or failure — returns JSON. An uncaught error must be caught by a global JSON error handler, never allowed to fall through to a framework's default HTML error page.

4.2 SHOULD HAVE (Retention & Monetization)

S.1 Accounts, Plans & Dev-Bypassed Billing

User Experience: Fledgling (free) → Songbird → Mockingbird → Lyrebird (custom), each raising daily build limits and, from Mockingbird up, marketplace-seller access.

Functional Mechanics

- BILLING_ENABLED=false (default) — plan changes apply instantly and nothing is charged, so the product is fully testable before a real payment provider is wired in.
- Password storage: scrypt hash only, never plaintext, verified with a timing-safe comparison.

S.2 Marketplace

User Experience: A Mockingbird+ seller lists an automation; a buyer purchases it and gets the automationId to run.

Functional Mechanics

- Platform fee scales down as order size grows (20% under 500 BDT, down to 7% over 10,000 BDT), with a further seller-tier discount — never a flat fee regardless of price.
- Every purchase and payout is written as an immutable Transaction record.

S.3 Durable, Provider-Agnostic Persistence

User Experience: Every account, recording, automation, run, and marketplace record survives a redeploy.

Functional Mechanics

- A single POSTGRES_URL (or DATABASE_URL) env var switches ALL storage from local JSON files to Postgres — same code path, any standard provider (Supabase, Neon, Vercel Postgres, Railway, RDS).
- Schema creation is idempotent (CREATE TABLE IF NOT EXISTS) and runs once at boot.
- A failed database connection at boot must degrade to “the app is up, /api/health reports it clearly” — never to “the whole process exits.”

S.4 Booking Filters & Currency

User Experience: A checked filter or a chosen currency in the recording is honored on every subsequent run — not silently dropped just because the site was replayed via a direct URL instead of clicks.

Functional Mechanics

- Recorded boolean filters (5-star, breakfast included, free cancellation, etc.) map to Booking's own nflt URL codes and are applied on every deterministic run — not silently dropped.
- A recorded currency choice maps to selected_currency on adapter-covered services.

4.3 COULD HAVE (Polish)

C.1 Reliable Email Connector

User Experience: A “send email” automation sends over the user's own SMTP account instead of replaying Gmail's fragile compose UI.

C.2 Encrypted Credential Vault

User Experience: A recorded login's password is never stored in the recording itself — it is supplied at run time and, if the user opts in, stored AES-256-GCM-encrypted at rest.

C.3 Packaged Extension Download

User Experience: A pre-built extension zip is published as a versioned, re-uploadable release asset so “get the extension” is a real download, not a link to source code the user has to build themselves.

4.4 WON'T HAVE (Explicit Hard Boundaries)

These are negative instructions for both human and agent contributors — they exist because each one was a real failure mode encountered while building this product, not a hypothetical.

ID	Boundary	Why it is a hard stop
W.1	No auto-solving CAPTCHAs or bypassing anti-bot systems	Assisted mode hands control to a human in a visible window; the product escalates to a person, it never attempts to defeat a site's bot defenses programmatically.
W.2	No reverse-engineering a private/session-bound API as a shortcut to a deterministic adapter	Confirmed directly: Booking's cars results API and GoZayaan's place-lookup are session-bound and return empty when called outside a real browser session. Building on top of that would silently break the moment the vendor changes an internal endpoint. These services stay on browser replay.

ID	Boundary	Why it is a hard stop
W.3	No plaintext credentials, and no locally-generated encryption key on a stateless host	A key auto-generated to local disk resets on every restart on a platform with no persistent disk (e.g. Vercel) — silently making previously encrypted data permanently unreadable. The key must come from an environment variable in any such environment.
W.4	No process.exit() on a recoverable startup failure in a shared server process	On a platform that runs one process per deployment (not one per request), exiting on a transient dependency failure (e.g. a bad DB connection string) takes down every route for every user, not just the DB-dependent ones. Must degrade and report via health-check instead.
W.5	No bulk/glob file deletion under the data directory without an explicit, itemized confirmation	A time-based or wildcard delete cannot distinguish disposable test artifacts from real user data that happens to fall in the same window. Any deletion under the data directory must list exact filenames and get explicit go-ahead first.

5. Automated Execution Gates for AI Agents

An agent must treat these as pass/fail gates, not suggestions — a task is not complete until every gate that applies to the changed code returns a clean exit code.

5.1 Compilation & Type-Safety Gate

Each package must compile with zero errors before a change is considered done:

```
cd backend   && npx tsc --noEmit && npm run build
cd frontend  && npx tsc --noEmit && npm run build
cd extension && npx tsc --noEmit && npm run build
```

backend builds to dist/server.js (tsc -p tsconfig.build.json); frontend and extension both build to dist/ via tsc -b && vite build — the extension's dist/ is what gets loaded unpacked in Chrome.

5.2 Runtime Health Gate

After any change to storage, auth, or server boot sequence, the health endpoint is the source of truth — not “it compiled”:

```
GET /api/health
# local, no POSTGRES_URL set:
{"ok": true, "service": "mimic-backend", "storage": "file"}
# production, database attached and reachable:
{"ok": true, "service": "mimic-backend", "storage": "postgres", "dbConnected": true}
```

`dbConnected: false` or an HTTP 503 means the gate fails even if every route compiled — the agent must not report the task done in that state.

5.3 Adapter Non-Regression Gate

Because deterministic adapters are the highest-trust path in the product, any change to `siteAdapters.ts` must be verified against a live run for every adapter it could affect, not just the one being changed — confirm the resulting URL and a non-zero result count for Booking hotels, Booking flights, Booking attractions, and GoZayaan flights before considering the change safe.

5.4 Local Instruction File (CLAUDE.md excerpt)

The following lives at the repository root and is loaded automatically by the agent on every session:

```
# Engineering Rules for Mimic

- Date math: NEVER use `date.toISOString()` to build a calendar-date string — it
  converts to UTC and can shift the date by a day. Build from local
  getFullYear()/getMonth()/getDate() parts directly.

- Storage: every store (recordings, automations, runs, users, sessions, api keys,
  credentials, listings, transactions) must keep working identically whether
  POSTGRES_URL is set or not. Never add a field to one backend without the other.

- Server boot: never process.exit() on a failed dependency (DB, etc.) in
  server.ts — log and continue; let /api/health report the real state.

- Error responses: every Express route must resolve to JSON, success or failure.
  A global error-handling middleware is required so an uncaught rejection never
  surfaces as an HTML page to the frontend.

- Adapters: build() returns null (never a best-guess URL) when an input can't be
  confidently resolved — the caller falls back to generic replay.

- Data safety: never bulk/glob-delete under backend/data/. List exact filenames
  and get explicit confirmation first, every time.
```

6. Execution Command Example for the AI CLI

When a contributor is ready to extend Mimic against this document, the unified command is issued directly in the project terminal:

```
claude "Read prd.md completely to map the system boundaries and MoSCoW  \
  gates. Implement the next MUST/SHOULD item, then verify by running       \
  npx tsc --noEmit and npm run build in every package touched, and confirm \
  GET /api/health reports dbConnected: true before considering it done."
```

Mimic — record once, replay forever.